# Project Sprint #5

**GitHub link: https://github.com/hafsaahmed05/CS449_Solitaire**

The main tasks of this assignment are:

(1) Conducting a code review exercise.
(2) Conduct review of your tests.
(3) Summarizing the lessons learned from Sprint 0 through Sprint 5.

**Total points: 20**

## 1. Code Review (8 points)

Apply source code review to one or two most important classes (and other classes if time permits) and report the findings. In addition to looking for bugs, the review should check: (1) whether the entire project has followed the coding standard in a consistent manner, (2) whether the project has followed the design principles introduced in class, and (3) whether there are code smells that indicate the need for refactoring. The following checklists provide basic guidelines. You may add new items to each of the checklists.

Make sure your answers resulted from the code review exercise. If there is no finding for an entry, you should provide an explanation. For example, if your answer to "Are the naming conventions violated?" is no, you should describe a naming convention and present an example. You will receive no credit for this part if your answers are simply yes or no without additional information.

**Classes that have been reviewed:**
SolitaireController
SolitaireModel

**Date/time duration of the code review exercise:**
04/23/2026 /~2-3 hours

| Checklist | Checklist Item | Findings |
|---|---|---|
| Coding Standards | Are the naming conventions violated? | No violations found. The project consistently uses:<br>• camelCase for variables/methods (selectedRow, makeMove)<br>• PascalCase for classes (SolitaireController, DiamondModel) |
| | Is the ordering convention of method arguments violated? | Mostly consistent: (row, col) ordering is preserved across:<br>• controller<br>• model<br>• view<br>This consistency reduces confusion and bugs. |
| | Any comments meaningless or inconsistent with the code? | Some comments are helpful, but a few are:<br>• too high-level<br>• not explaining why, only what<br><br>**Improvement**: Add reasoning comments (e.g., why threading is used in autoplay) |
| | Any code block has an inconsistent formatting style? | Formatting is consistent:<br>• Proper indentation<br>• Clear spacing between sections<br>• Logical grouping using comment blocks |
| Design Principles | Any class/method not well modularized? | Good modular separation:<br>• Model → game logic<br>• View → UI<br>• Controller → coordination |
| | Any class with poor abstraction? | Strong abstraction in SolitaireModel:<br>• Shared logic implemented once<br>• Subclasses only define board shape<br><br>However, some logic (like directions) is overridden in HexagonModel, which slightly increases complexity but is necessary for change in movement for the HexagonModel. |
| | Is the visibility of any variable, method, and class inappropriate? | Mostly appropriate: Internal fields are private or protected Public methods expose only necessary behavior<br><br>Example:<br>`protected int[][] board;`<br><br>Possible improvement: board could be more encapsulated (avoid direct subclass dependence) |
| | Is design by contract (pre/postcondition) violated? | Partially implemented: Methods assume valid inputs but don't enforce them explicitly |

| | | |
|---|---|---|
| | | Example:<br>`public boolean makeMove(...)`<br><br>Relies on isLegalMove() instead of validating parameters early.<br><br>**Improvement**: Add explicit checks or documentation of preconditions |
| | Is the Open-Closed Principle violated? | Mostly satisfied: New board types (Diamond, Hexagon) added without modifying base logic<br><br>Example:<br>`private SolitaireModel buildModel(String type, int size)`<br><br>**Weakness**: Uses switch → not fully extensible<br>**Improvement**: polymorphism or factory pattern |
| | Is the Single Responsibility Principle violated? | Mostly followed, but SolitaireController is doing too much:<br>• UI coordination<br>• Game logic triggering<br>• File validation<br>• Replay logic<br><br>This violates SRP slightly. |
| Code Smells | Are there magic numbers? | Found in multiple places:<br><br>Example:<br>`Thread.sleep(300);`<br>`Thread.sleep(400);`<br><br>**Fix**: Replace with named constants |
| | Are there unnecessary global / class variable? | **1. SIZE_CONST in SolitaireModel**<br>`public static final int SIZE_CONST = 7;`<br><br>Issue:<br>Not consistently used across the project<br>Board size is already dynamic<br><br>Why it's unnecessary:<br>Could create confusion between fixed vs dynamic size<br><br>**2. Controller selection state**<br>`private int selectedRow = -1;`<br>`private int selectedCol = -1;` |

| | |
|---|---|
| | Issue:<br>Stored globally in controller<br>Only used temporarily for user interaction<br><br>Why it's questionable:<br>Could be encapsulated into a small object or handled differently<br><br>**3. Recorder always instantiated**<br>`private final GameRecorder recorder = new GameRecorder();`<br><br>Issue:<br>Exists even when recording is not used<br><br>Improvement:<br>Could be lazily initialized when recording starts |
| Is there duplicate code? | Replay + autoplay both:<br>• loop through moves<br>• sleep<br>• update UI<br><br>**Improvement**: Could be refactored into a shared helper method. |
| Are there long methods? | onReplay() is very long and does:<br>• file validation<br>• parsing<br>• model rebuilding<br>• replay execution<br><br>**Improvement**: This is a clear refactor candidate. |
| Is there any long parameter list? | Not a major issue, but:<br>`recordMove(int fromRow, int fromCol, int toRow, int toCol)`<br><br>**Improvement**: Could be replaced with a Move object. |
| Is there over-complex expression? | Replay validation has many checks:<br>`if (!GameRecorder.isBoardType(line)`<br>`&& !GameRecorder.isBoardSize(line)`<br>`...`<br>`)`<br><br>**Improvement**: Hard to read → could be simplified. |
| Is there switch or if-then-else<br>that needs to be replaced with polymorphism | This is a key issue:<br>`switch (type) {`<br>`case "Hexagon": ...`<br>`case "Diamond": ...`<br>`}` |

| | |
|---|---|
| | Should be replaced with:<br>Factory pattern or polymorphic instantiation |
| Any variable or method name whose intent is unclear? | `int[] moveCoords = model.getNextMove();`<br><br>**Issue:**<br>• moveCoords is vague<br>• Doesn't communicate structure (from/to)<br><br>Better:<br>`int[] move = {fromRow, fromCol, toRow, toCol};`<br>or use a Move object<br><br>---<br>Example from SolitaireModel:<br>`int g = getGridSize();`<br><br>Issue:<br>• g is not descriptive<br>• Reader has to infer it means "grid size"<br><br>Better:<br>`int gridSize = getGridSize();`<br>`public boolean makeAnyMove()`<br><br>---<br>`public boolean makeAnyMove()`<br><br>Issue: Name doesn't clearly indicate behavior It both finds AND executes a move<br><br>Why this matters: Could be confused with a method that just checks for moves<br><br>Better: executeFirstValidMove() or split into:<br>findNextMove(), applyMove() |
| Any similar methods in different classes? | Yes, small duplication pattern<br>1. Board shape logic repeated across models<br><br>Each model defines its own:<br>`public boolean isValidCell(int row, int col)` |

Files:

- EnglishModel
- DiamondModel
- HexagonModel

Observation: These methods are similar in purpose but implemented differently This is intentional polymorphism, not bad duplication

This is GOOD design:

- Each subclass defines its own board shape
- Follows Open-Closed Principle

2. Direction logic duplication (minor concern)
SolitaireModel:

```
protected int[][] getDirections()
```

HexagonModel overrides it:

```
protected int[][] getDirections()
```

Issue:
Similar structure, but extended directions
Slight duplication

Acceptable because:
Diagonal movement rules for the HexagonModel require override

| Bugs | Buggy code snippet | What is the bug? | Why is it a bug? |
|---|---|---|---|
| | In SolitaireController.java:<br><br>`int[] moveCoords = model.getNextMove();`<br>`if (moveCoords == null) break;`<br><br>`boolean moved = model.makeAnyMove();`<br>`if (!moved) break;`<br><br>`recorder.recordMove(moveCoords[0], moveCoords[1], moveCoords[2], moveCoords[3]);` | getNextMove() and makeAnyMove() both independently scan the board for the first valid move.<br><br>The fix is to have makeAnyMove() return the move it executed, or just use getNextMove() and then call makeMove() directly with those coordinates. | They are not guaranteed to return the same move. If the board state somehow changes between the two calls, unlikely in single-threaded logic but architecturally fragile, the recorded move won't match what was actually executed. |

| | | |
|---|---|---|
| randomizeBoard() | randomizeBoard() can produce a board with no valid moves but doesn't set game over.<br>In SolitaireModel.java, randomizeBoard() randomly assigns pegs and empties, only guaranteeing one empty cell at the center. | It's entirely possible to produce a board where no legal moves exist, but the game continues normally without notifying the player. The controller only checks isGameOver() after makeMove() is called, never after onRandomize(). So the player would be stuck with no moves and no feedback. |

**2.** Review how adequately you have tested your code and discuss how could you improve your tests based on lessons learned in the class **(8 points).**

Do not simply answer "yes" or "no". Give examples or the process you followed for "yes" answers. Concurrently, for "no" answers, discuss reasons and possible improvement.

a. **Have you performed acceptance tests for all user stories?**

- Yes, acceptance testing was performed for all core user stories through automated JUnit tests.
- The test class SolitaireModelTest directly maps to user stories US1–US10, covering:
  - board size and type selection
  - starting a new game
  - manual and automated moves
  - game-over and win conditions
  - randomization
  - recording and replay functionality
- Each user story includes multiple acceptance criteria. This was verified using test cases.
- In addition to automated tests, manual testing was also performed through the UI to verify interaction-based features such as clicking cells, autoplay, and replay visualization.

b. **Have you considered positive, negative and malicious scenarios in the acceptance tests?**

- Yes, multiple types of scenarios were tested:
  - Positive scenarios
    - Valid moves → makeMove() returns true
    - Successful autoplay sequences
    - Correct replay behavior
  - Negative scenarios
    - Illegal moves – empty source, occupied destination, diagonal moves
    - No-move situations – makeAnyMove() returns false
    - Invalid board states – tested using helper boards
  - Malicious scenarios
    - Invalid replay files – missing BOARD_TYPE
    - Corrupted or incomplete data during replay parsing

- While malicious testing is partially covered, additional extreme cases (like heavily corrupted files or unexpected formats) could still be explored. However, the code inherently does have a check for not allowing corrupted files.

c. **Have you written unit tests (beyond the acceptance tests) for the implemented features?**
- Yes, a lot of unit tests beyond basic acceptance tests
- The test suite includes –
    - Validation of game logic – isLegalMove, makeMove, isGameOver
    - Consistency checks like peg count vs board state
    - Board-specific behavior for all models – English, Diamond, Hexagon
    - Helper-generated edge cases such as winning boards, stuck boards with no legal moves
- These tests isolate the model logic and ensure correctness independent of the UI.

d. **What is the code coverage obtained for testing the main features?**
- The code coverage is high for the model layer since –
    - All major methods are tested
    - Multiple edge cases and scenarios are covered
    - All user stories are validated through tests
- Coverage is lower for controller logic (UI-driven behavior) and view components, since they were tested manually

e. **Have you added tests to improve code coverage?**
- Yes, additional tests were added to improve coverage
    - Edge-case board configurations
    - Multiple board types
    - Replay and recording validation
    - Randomized board behavior

f. **Have you performed any other automated tests e.g., fuzzing?**
- No, formal fuzzing was implemented
- However, randomized testing was achieved through repeated calls to randomizeBoard() and validation that board states remain consistent after randomization
- **Improvement**: fuzz testing could be added for replay file parsing and randomize board generation to improve.

g. **How do you improve your current testing of the code, especially the testing of the game logic?**
- To further improve testing –
    - Add controller-level tests using mocks to simulate UI interaction
    - Expand malicious input testing for replay files
    - Introduce automated replay validation ensuring recorded games always reproduce correctly
    - Apply fuzz testing to stress-test board generation and parsing logic
    - Increase integration testing across the move, view and controller

**3. Summarize the lessons learned from the entire project by answering the following questions from the perspectives of development processes, coding, design, refactoring, and testing (4 points):**

- **What did you personally gain from the project?**

Through this project, I gained valuable experience in developing a full, multi-sprint software application rather than just small, isolated programs. This was one of the first times I worked on an extensive project that required long-term planning, incremental development, and continuous improvement across multiple stages.

I also learned how to properly refactor code to make it more organized and readable. As new features were added, I began to recognize the importance of restructuring code to maintain clarity and avoid writing overly complex methods and functions. This helped me better understand how to write maintainable code that can be extended without introducing complexity.

Another major takeaway was understanding how the sprint process works in practice. Breaking the project into multiple sprints allowed me to focus on smaller, manageable goals while still contributing to a larger system. I also learned how to incrementally develop features and reflect on previous work to improve future implementations.

Additionally, I gained experience working with user stories and acceptance criteria. Mapping features to specific user stories helped guide development and ensured that each requirement was clearly defined and testable. This will be very helpful in a real-world environment for understanding client requirements and fulfilling them.

- **What does your project do well, and what could your project do better?**
  The project does really well in implementing all major components of the Solitaire game. Core functionality such as board setup, move validation, game progression, autoplay, randomization, and replay all work correctly.

  The system also supports multiple board types, which demonstrates flexibility and effective use of object-oriented design. Another strength of the project is its structured testing. The use of unit tests mapped to user stories ensures that key features are validated and that the game logic behaves as expected across different scenarios.

  However, there are areas where the project could be improved. While testing is strong for the model layer, additional testing could be done for the controller and overall system integration. Some parts of the code, particularly in the controller, could also be simplified and further refactored to reduce complexity and improve maintainability. There could also be more done on the validation side

- **How could you improve your development process if you develop a similar game from scratch?**
  If I were to develop a similar game from scratch, I would improve my development process by breaking the project into more than five sprints and making the increments even more gradual. Having more checkpoints would allow for better tracking of progress and earlier identification of design or implementation issues.

  I would also focus on planning and breaking down the architecture more and therefore, the testing strategy earlier in the process. This includes writing tests alongside development ensuring that all components, including the controller and integration points, are thoroughly tested.

  Additionally, I would refactor more consistently throughout development rather than waiting until later stages. This would help keep the codebase clean and prevent complexity from building up over time.